
UNIVERSIDAD AUTÓNOMA DE SANTO DOMINGO
INF-5250 — INGENIERÍA DE SOFTWARE II

SISTEMA360

Entregable I (E1) — Actividad 1

Informe de cálculo de métricas del software — Versión final

INTEGRANTES

Jean Carlos Farias **100362416**
Albert Encarnacion **100634972**
Kevin Daniel Batista **100575671**
Miguel Elias Ramirez **100453980**

Proyecto: Sistema360
Rama: main · Commit: d467701 · Tag: v0.1-E1-A1
Fecha: 02/10/2026

Santo Domingo, República Dominicana

Tabla de contenido

Tabla de contenido	2
1. Portada	3
2. Contexto del sistema medido	3
3. Herramienta(s) utilizada(s)	3
4. Evidencia de pasos dados antes del proceso de cálculo de métrica	3
— A) Inicialización del repositorio local y publicación en plataforma remota	3
— B) Congelar una versión del código a medir	3
<i>Evidencias de la versión congelada</i>	3
5. Resultados de métricas de tamaño	4
6. Resultados de métricas orientadas a objetos	4
7. Interpretación de resultados	6
— 1. ¿El tamaño (LOC/KLOC) es razonable para el alcance actual del MVP, según lo definido en la SRS?	6
— 2. ¿El número de clases refleja una separación de responsabilidades adecuada, o hay muy pocas clases “gigantes” (síntoma de baja cohesión)?	6
— 3. ¿El grado de acoplamiento es alto o bajo?	7
— 4. ¿Qué relación existe entre estas métricas y la productividad del equipo?	7
8. Conclusiones y próximos pasos	7
9. Fuentes consultadas	8
10. Anexos	8
— Anexo A. Salida actualizada de cloc v2.10	8
— Anexo B. Conteo e identificación de las clases propias	8
— Anexo C. Verificación del acoplamiento directo	9
11. Bibliográficas	9

1. Portada

La portada se encuentra al inicio del documento, con los integrantes del equipo y los datos de referencia del repositorio analizado.

2. Contexto del sistema medido

Sistema360 es el sistema base del proyecto que será medido en esta actividad. El análisis se realizará sobre una versión específica del código almacenada en GitHub, de modo que las métricas sean reproducibles y correspondan a un estado exacto del proyecto.

Repositorio del proyecto: <https://github.com/Juampiezas/sistema360>

Versión de referencia actual: rama main, commit corto d467701, con la etiqueta v0.1-E1-A1.

3. Herramienta(s) utilizada(s)

Para las métricas orientadas al tamaño se utilizó cloc (Count Lines of Code), versión 2.10, ejecutado desde la raíz del repositorio con el comando `cloc . --exclude-dir=node_modules,venv,vendor,.git`.

Para el conteo de clases y la verificación de dependencias directas entre las clases propias se utilizaron comandos de PowerShell, excluyendo las carpetas vendor y dompdf para evitar contabilizar código de terceros.

4. Evidencia de pasos dados antes del proceso de cálculo de métrica

— A) Inicialización del repositorio local y publicación en plataforma remota

El proyecto ya cuenta con repositorio Git y publicación remota en GitHub. La rama principal utilizada es main y el repositorio remoto se encuentra actualizado.

- Configuración del repositorio local: realizada.
- Publicación en plataforma remota: realizada en GitHub.
- Colaboración e integración del equipo: pendiente de documentar con la evidencia correspondiente.

— B) Congelar una versión del código a medir

Antes del cálculo de métricas se verificó que el repositorio estuviera limpio y se corrigió la etiqueta v0.1-E1-A1 para que correspondiera a la versión actual que será utilizada como referencia.

Evidencias de la versión congelada

La versión congelada queda identificada por el commit d467701 y la etiqueta v0.1-E1-A1, visibles en la barra de estado de las capturas de la Sección 5 y en el Anexo A.

5. Resultados de métricas de tamaño

Se ejecutó cloc v2.10 sobre la versión congelada de Sistema360. La herramienta analizó 38 archivos únicos y reportó 1,697 líneas en blanco, 200 líneas de comentarios y 3,658 líneas de código fuente (SLOC).

Para este informe se considera SLOC como la columna “code” de cloc. El LOC físico total se obtiene sumando líneas en blanco, comentarios y código: $1,697 + 200 + 3,658 = 5,555$ LOC. Por tanto, KLOC = $5,555 / 1,000 = 5.555$ KLOC.

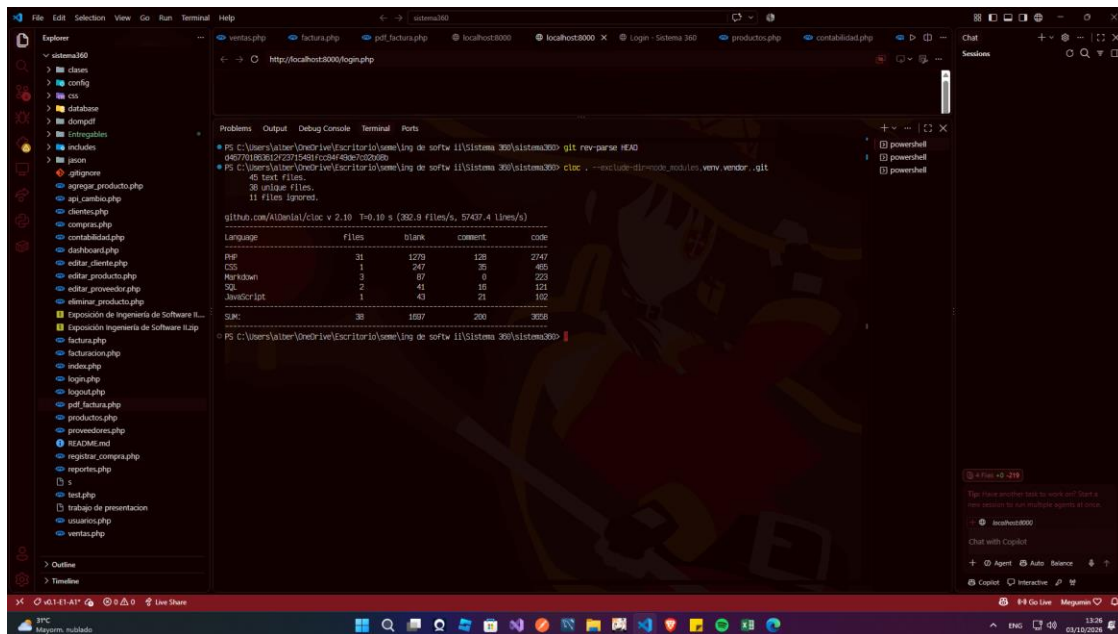


Figura 1. Salida de cloc v2.10 para Sistema360 — ventana completa de VS Code, con commit, ruta del proyecto y fecha/hora visibles.

Métrica	Resultado
LOC físico	5,555
SLOC	3,658
KLOC	5.555

Tabla 1. Resumen de métricas de tamaño.

6. Resultados de métricas orientadas a objetos

Número total de clases propias del sistema: 5. Se incluyeron exclusivamente las clases desarrolladas para Sistema360 y se excluyeron las dependencias externas ubicadas en vendor y dompdf.

Clases identificadas: Cliente, Producto, Proveedor, Usuario y Venta.

Página 5 de 10

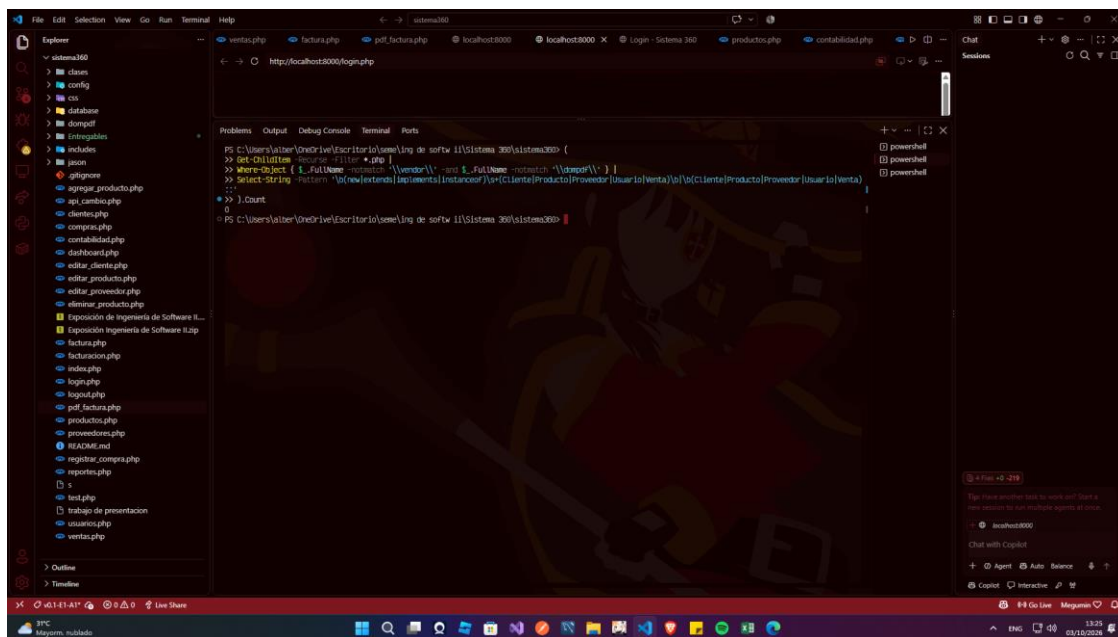


Figura 3. Verificación del grado de acoplamiento directo entre las clases propias de Sistema360.

7. Interpretación de resultados

La interpretación se realiza con base en los resultados obtenidos para la versión congelada de Sistema360: 5,555 LOC físicos, 3,658 SLOC, 5.555 KLOC, 5 clases propias y un acoplamiento directo promedio de 0.00 entre dichas clases.

— 1. ¿El tamaño (LOC/KLOC) es razonable para el alcance actual del MVP, según lo definido en la SRS?

Sistema360 presenta 5,555 líneas físicas de código (5.555 KLOC) y 3,658 líneas de código fuente (SLOC). Para el alcance funcional actual —clientes, productos, proveedores, usuarios, ventas, compras, facturación, contabilidad y reportes— el volumen puede considerarse moderado y coherente con un MVP funcional. Sin embargo, el proyecto no contiene una SRS formal dentro de los archivos revisados, por lo que esta valoración se realiza con base en la descripción funcional disponible en el README. El tamaño por sí solo no determina la calidad: también importa cómo está distribuida la lógica y qué tan fácil resulta modificarla.

— 2. ¿El número de clases refleja una separación de responsabilidades adecuada, o hay muy pocas clases “gigantes” (síntoma de baja cohesión)?

Se identificaron 5 clases propias: Cliente, Producto, Proveedor, Usuario y Venta. Estas clases son pequeñas y representan entidades concretas del dominio, por lo que no se observa el problema de “clases gigantes” dentro de este grupo. No obstante, el sistema posee más módulos funcionales que clases: compras, facturación, contabilidad y reportes se implementan principalmente mediante archivos PHP procedimentales. Por ello, la separación de responsabilidades orientada a objetos es parcial. El diseño podría mejorar incorporando clases o servicios específicos para esas funciones y separando con mayor claridad la lógica de negocio, el acceso a datos y la presentación.

— 3. ¿El grado de acoplamiento es alto o bajo?

El acoplamiento directo medido entre las 5 clases propias es bajo: se detectaron 0 dependencias directas y el promedio calculado fue $0 \div 5 = 0.00$. Esto indica que Cliente, Producto, Proveedor, Usuario y Venta no dependen entre sí mediante new, extends, implements, instanceof o llamadas estáticas directas. Este resultado favorece el aislamiento de esas clases y reduce el riesgo de que un cambio en una de ellas afecte automáticamente a las demás. Sin embargo, esta métrica no representa todo el acoplamiento del sistema: varios módulos PHP acceden directamente a la base de datos y algunos utilizan librerías externas, por lo que existen dependencias arquitectónicas que no aparecen en el valor 0.00 de acoplamiento entre clases.

— 4. ¿Qué relación existe entre estas métricas y la productividad del equipo?

Las métricas muestran dos efectos distintos sobre la productividad. Por un lado, el bajo acoplamiento directo entre las 5 clases facilita modificarlas de forma independiente. Por otro, la escasa separación orientada a objetos en varios módulos hace que parte de la lógica de negocio, las consultas SQL y la presentación estén concentradas en archivos PHP de cada pantalla. Esto puede aumentar el tiempo necesario para localizar cambios, probar modificaciones y evitar regresiones. Para mejorar la productividad en las próximas entregas, conviene mantener el bajo acoplamiento entre clases y, al mismo tiempo, trasladar progresivamente la lógica repetida o sensible a componentes reutilizables y claramente separados.

8. Conclusiones y próximos pasos

Los resultados obtenidos permiten establecer una línea base técnica del estado actual de Sistema360. El proyecto presenta un tamaño de 5,555 LOC físicos, 3,658 SLOC y 5.555 KLOC, distribuidos principalmente en archivos PHP. Además, se identificaron 5 clases propias del sistema y un acoplamiento promedio directo de 0.00 entre ellas, según el método de búsqueda aplicado.

Estos resultados muestran que el sistema posee una estructura funcional y un volumen de código manejable para su alcance actual. Sin embargo, el análisis también evidencia que una parte importante de la funcionalidad está implementada directamente en archivos PHP y no necesariamente concentrada en clases, por lo que la estructura orientada a objetos aún puede fortalecerse.

Como próximos pasos, antes de continuar con nuevas funcionalidades, se recomienda mantener la separación entre las clases existentes, evitar generar dependencias innecesarias entre ellas y continuar organizando la lógica del sistema en componentes con responsabilidades claramente definidas. También será importante conservar la misma versión base identificada mediante Git para poder comparar la evolución de las métricas en las próximas entregas.

Con esta medición, el equipo dispone de una referencia inicial que permitirá evaluar posteriormente si el crecimiento del sistema mejora o deteriora aspectos como la mantenibilidad, la modularidad y la productividad del desarrollo.

9. Fuentes consultadas

Para la elaboración de este informe se utilizaron como fuentes directas la guía del profesor correspondiente al Entregable I (E1) / Actividad 1 de INF-5250 Ingeniería de Software II, el repositorio del proyecto Sistema360 en GitHub y las salidas generadas por las herramientas y comandos ejecutados sobre la versión congelada del código.

- Guía del profesor: Actividades entregables Ingeniería de Software II, Entregable I (E1) / Actividad 1.
- Repositorio del proyecto Sistema360: <https://github.com/Juampiezas/sistema360>
- Salida de cloc v2.10 obtenida sobre Sistema360 para las métricas de tamaño.
- Resultados de PowerShell utilizados para el conteo de clases y la verificación del acoplamiento directo.

10. Anexos

A continuación se presentan únicamente las evidencias que respaldan directamente las métricas reportadas en E1-A1. Se eliminaron las capturas recortadas o repetidas que no aportaban información adicional. Las capturas conservadas muestran la ventana completa de Visual Studio Code, la ruta del proyecto y la fecha/hora visible del equipo, manteniendo la misma versión base v0.1-E1-A1 y el mismo commit analizado.

— Anexo A. Salida actualizada de cloc v2.10

Con commit, ruta del proyecto y fecha/hora visibles.

```
PS C:\Users\alber\OneDrive\Escritorio\workspace\inf-5250\Software II\Entregable I (E1)\Actividad 1\Sistema360> git rev-parse HEAD
d497118361272715481fcd4449e7320208
PS C:\Users\alber\OneDrive\Escritorio\workspace\inf-5250\Software II\Entregable I (E1)\Actividad 1\Sistema360> cloc . --exclude-dir=node_modules,*.vs,*.vscode,*.git
<5 text files.
36 unique files.
11 files ignored.
```

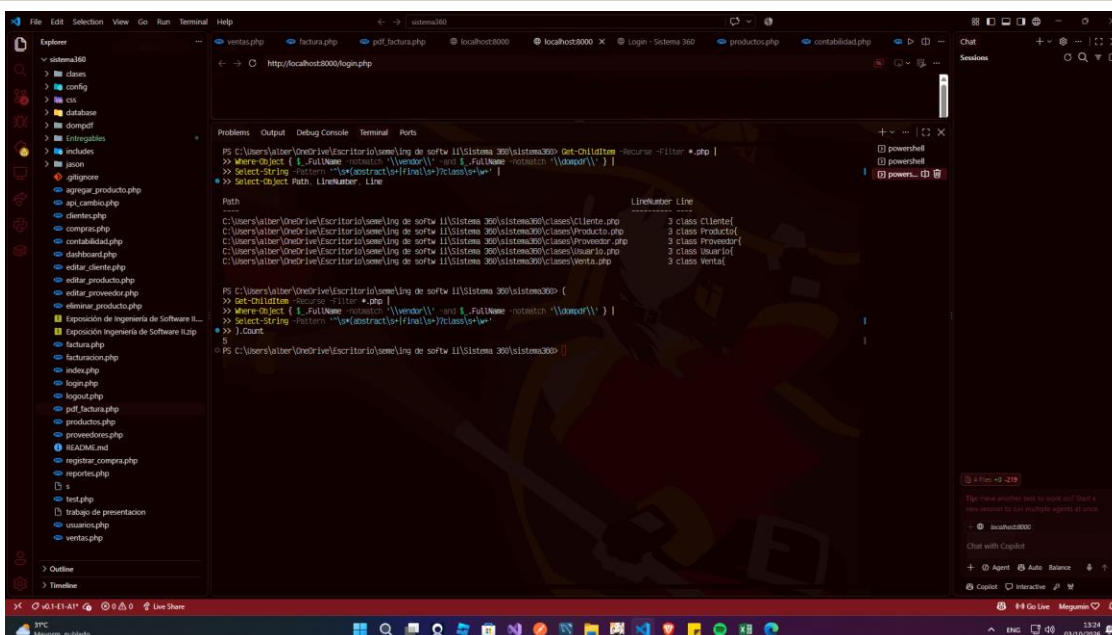
Language	Files	Blank	Comment	Code
PHP	31	1279	128	2747
CS	1	247	25	405
Markdown	3	67	0	223
SQL	2	41	10	121
JavaScript	1	43	24	102
SLOC	38	1557	200	3598

```
PS C:\Users\alber\OneDrive\Escritorio\workspace\inf-5250\Software II\Entregable I (E1)\Actividad 1\Sistema360>
```

Anexo A. Salida actualizada de cloc v2.10, con commit, ruta del proyecto y fecha/hora visibles.

— Anexo B. Conteo e identificación de las clases propias

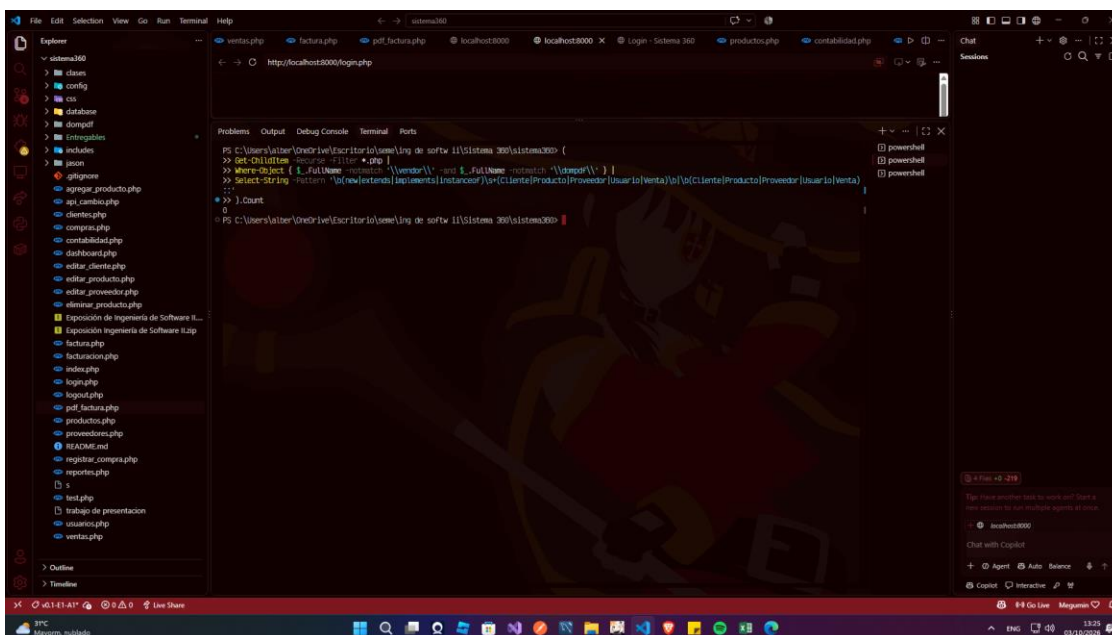
Actualizado, de las 5 clases propias de Sistema360.



Anexo B. Conteo e identificación actualizada de las 5 clases propias de Sistema360.

— Anexo C. Verificación del acoplamiento directo

Actualizada, entre las clases propias del sistema.



Anexo C. Verificación actualizada del acoplamiento directo entre las clases propias del sistema.

11. Bibliográficas

No se incorporaron obras bibliográficas adicionales para sustentar resultados numéricos que fueron obtenidos directamente del código y de las herramientas ejecutadas. La referencia académica utilizada para definir la estructura, las métricas y el procedimiento fue la guía del profesor de INF-5250.

Si el equipo consulta posteriormente libros o estándares adicionales para ampliar la interpretación, deberán agregarse aquí con su referencia completa.